A2: Command Line Parsing

This guide shows the small amount of command-line parsing machinery needed for A2.

Create the parser and options

It will be helpful to open the Batteries option parser to avoid having to qualify the module names:

```
open BatOptParse
```

Create a parser with the required usage line and a short description:

```
let parser =
  OptParser.make
    ~usage:
      "%prog MARKET.csv STRATEGY OUTPUT.csv --cash DOLLARS [--hold SYMBOL]"
    ~description:"Backtest one strategy against historical prices."
    ()
```

Both A2 options begin as strings. Meaning-specific checks happen after parsing.

```
let cash = StdOpt.str_option ~metavar:"DOLLARS" ()
let hold = StdOpt.str_option ~metavar:"SYMBOL" ()

let () =
  OptParser.add parser ~long_name:"cash" ~help:"positive initial cash" cash;
  OptParser.add parser ~long_name:"hold"
    ~help:"security bought by buy-and-hold" hold
```

`OptParser.make` automatically installs `-h` and `--help`. Either flag prints the generated usage message and exits successfully before your program opens a market file.

Parse positional and named values

Call the parser once:

```
let positional = OptParser.parse_argv parser
```

For a valid command, `positional` contains the market file's name, one strategy name, and the output filename. Check that it has exactly three elements.

Use `Opt.opt` to inspect a named option:

```
let cash_text = Opt.opt cash
let hold_text = Opt.opt hold
```

Each result is a `string option`. `None` means the user did not supply it. The parser itself reports unknown option names and options missing their following value.

Validate in two phases

Before opening the market file, check facts that do not depend on its contents:

- exactly three positional arguments;
- a known strategy name;
- a supplied `--cash` value in the required positive `DOLLARS` form.

In the event of an error, call `OptParser.error` with a specific message. It prints the usage information and error message, then exits unsuccessfully. `OptParser.usage` only prints the usage information and returns.

After opening the market file, initialize the selected strategy. Buy-and-hold can then check that `--hold` was supplied and exactly matches one security named in the file. Cash and the default momentum strategy ignore `--hold` if it was supplied. If you provide your own stateful strategy, it can use its own custom options and check any additional facts that depend on the file's contents.

For additional library details, see the Batteries documentation for `BatOptParse` **(https://ocaml-batteries-team.github.io/batteries-included/hdoc2/BatOptParse.html)**, `OptParser` **(https://ocaml-batteries-team.github.io/batteries-included/hdoc2/BatOptParse.OptParser.html)**, `StdOpt` **(https://ocaml-batteries-team.github.io/batteries-included/hdoc2/BatOptParse.StdOpt.html)**, and `Opt` **(https://ocaml-batteries-team.github.io/batteries-included/hdoc2/BatOptParse.Opt.html)**.